



Python

Métodos 'mágicos'

Prof. Edson José Pacheco

2026

Objetivos de Aprendizagem: Protocolos e Métodos Mágicos

Identidade de Objeto

- ▶ Implementar representações com `__str__` e `__repr__`.
- ▶ Controlar a criação de instâncias com `__new__`.
- ▶ Gerenciar o ciclo de vida e destruição de objetos.

Sobrecarga de Operadores

- ▶ Habilitar operações matemáticas (`__add__`, `__sub__`).
- ▶ Implementar comparadores lógicos (`__eq__`, `__lt__`).
- ▶ Criar classes numericamente compatíveis.

Protocolos Customizados

- ▶ Criar **Context Managers** (`__enter__` / `__exit__`).
- ▶ Implementar objetos iteráveis (`__iter__` / `__next__`).
- ▶ Simular comportamento de funções com `__call__`.

Estado e Atributos

- ▶ Customizar acesso a atributos (`__getattr__`).
- ▶ Utilizar **Data Classes** para gestão de estado.
- ▶ Implementar o protocolo de container (`__getitem__`).



CLIQUE PARA ABRIR NO NAVEGADOR

 Um *Overview* sobre Padrões de Objetos

Identidade de Objeto



O Python oferece dois métodos para converter um objeto em string, atendendo a propósitos distintos de comunicação.

Implementação Técnica:

Python

```
class Sensor:
    def __init__(self, id, valor):
        self.id = id
        self.valor = valor

    def __str__(self):
        # Representação amigável (Usuário)
        return f"Sensor {self.id}: {self.valor}V"

    def __repr__(self):
        # Representação técnica (Desenvolvedor)
        return f"Sensor(id='{self.id}', valor={self.valor})"

s = Sensor("A1", 3.3)
print(str(s)) # Saída: Sensor A1: 3.3V
print(repr(s)) # Saída: Sensor(id='A1', valor=3.3)
```

Diferenças:

- ▶ `__str__`: Focado em **legibilidade**. Invocado por `print()` e `format()`.
- ▶ `__repr__`: Focado em **precisão**. Idealmente, deve permitir recriar o objeto.
- ▶ **Debug**: O console interativo e logs utilizam o `repr` por padrão.

O método `__new__` é um método estático que precede o `__init__`. Ele é utilizado quando é necessário controlar a **criação** física do objeto.

Padrão Singleton (Instância Única):

Python

```
class DriverI2C:
    _instancia = None

    def __new__(cls, *args, **kwargs):
        if not cls._instancia:
            # Cria a instância apenas uma vez
            cls._instancia = super().__new__(cls)
        return cls._instancia

# Ambas as variáveis apontam para o mesmo objeto
d1 = DriverI2C()
d2 = DriverI2C()
print(d1 is d2) # Saída: True
```

Aplicações Avançadas:

- ▶ **Singletons:** Garantir que periféricos de hardware tenham apenas um driver ativo.
- ▶ **Imutáveis:** Customizar a criação de subclasses de tipos nativos como tuple ou int.
- ▶ **Meta-programação:** Alterar o objeto antes mesmo de sua inicialização.

O Python gerencia a memória automaticamente através do **Garbage Collector**. O método `__del__` é o finalizador invocado quando a contagem de referências chega a zero.

O Finalizador de Objeto:

Python

```
class ArquivoTemporario:
    def __init__(self, nome):
        self.nome = nome
        print(f"Arquivo {nome} aberto.")

    def __del__(self):
        # Limpeza de recursos externos
        print(f"Limpando {self.nome} da RAM.")

# Uso
temp = ArquivoTemporario("cache.bin")
del temp # Força a remoção da referência
```

Mecanismo:

- ▶ **Referência Zero:** O objeto é marcado para destruição quando ninguém mais o aponta.
- ▶ **Risco:** O momento exato da execução do `__del__` não é garantido pelo interpretador.
- ▶ **Boas Práticas:** Para recursos críticos (hardware), prefira Context Managers (with).

Protocolos Customizados



O comando **with** permite que um objeto gerencie seu próprio ciclo de vida, garantindo que recursos sejam liberados mesmo em caso de erro.

Implementação OO:

Python

```
class BombaAgua:
    def __enter__(self):
        print("Ligando Bomba...")
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Desligando Bomba com segurança.")
        # Retornar True ignora exceções internas
        return False

# Uso prático
with BombaAgua() as bomba:
    print("Irrigando plantaço...")
    # Se houver erro aqui, o __exit__ roda!
```

Funcionamento:

- ▶ **`__enter__`**: Configuração inicial e retorno do objeto.
- ▶ **`__exit__`**: Limpeza final. Recebe metadados de exceções para logs.
- ▶ **Aplicações**: I2C, SPI, Conexões de Rede e Arquivos.

Para que uma classe se comporte como uma lista ou sequência, ela deve implementar o protocolo de **Iterador**.

Criando uma Sequência:

Python

```
class ContadorHardware:
    def __init__(self, limite):
        self.limite = limite
        self.atual = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.atual < self.limite:
            res = self.atual
            self.atual += 1
            return res
        raise StopIteration

for n in ContadorHardware(3):
    print(n) # 0, 1, 2
```

Mecânica Interna:

- ▶ **`__iter__`**: Retorna o próprio objeto iterador.
- ▶ **`__next__`**: Retorna o próximo valor ou encerra com `StopIteration`.
- ▶ **Vantagem**: Economia de RAM ao gerar dados sob demanda.

O método `__call__` permite que uma instância de classe seja invocada como se fosse uma função comum, mantendo seu estado interno.

Simulando Funções:

Python

```
class Multiplicador:
    def __init__(self, fator):
        self.fator = fator

    def __call__(self, valor):
        return valor * self.fator

# Instanciação
dobrar = Multiplicador(2)

# Uso como função
print(dobrar(10)) # Saída: 20
```

Casos de Uso:

- ▶ **Filtros Dinâmicos:** Objetos que processam sinais com parâmetros fixos.
- ▶ **Decoradores:** Classes que envolvem outras funções com lógica extra.
- ▶ **Estado:** Diferente de funções, o objeto "lembra" de configurações prévias.

Sobrecarga de Operadores



Sobrecarga de Operadores: Operações Matemáticas

A sobrecarga permite que objetos utilizem operadores como + e - através dos métodos `__add__` e `__sub__`.

Exemplo: Vetor de Dados de Sensores

Python

```
class Leitura:
    def __init__(self, valor):
        self.valor = valor

    def __add__(self, outro):
        # Retorna uma nova instância com a soma
        return Leitura(self.valor + outro.valor)

    def __repr__(self):
        return f"Leitura({self.valor})"

# Operação intuitiva
s1 = Leitura(10)
s2 = Leitura(20)
s3 = s1 + s2
print(s3) # Saída: Leitura(30)
```

Funcionamento:

- ▶ `__add__`: Invocado quando o objeto está à esquerda do sinal de +.
- ▶ **Imutabilidade**: É boa prática retornar uma **nova instância** em vez de alterar a atual.
- ▶ **Legibilidade**: Transforma chamadas de métodos complexos em sintaxe matemática clara.

Sobrecarga de Operadores: Comparadores Lógicos

Para que o Python saiba como comparar dois objetos customizados, devemos implementar métodos como `__eq__` (igual) e `__lt__` (menor que).

Lógica de Comparação:

Python

```
class Sensor:
    def __init__(self, id, tensao):
        self.id = id
        self.tensao = tensao

    def __eq__(self, outro):
        # Igualdade baseada no ID e Tensão
        return self.id == outro.id and \
            self.tensao == outro.tensao

    def __lt__(self, outro):
        # Menor que baseado na Tensão
        return self.tensao < outro.tensao

s1 = Sensor("A", 3.3)
s2 = Sensor("B", 5.0)
print(s1 < s2) # Saída: True (lt invocado)
```

Protocolos de Ordem:

- ▶ `__eq__`: Define o comportamento do operador `==`.
- ▶ `__lt__`: Habilita o uso de `<` e permite que listas de objetos sejam **ordenadas** automaticamente via `sort()`.
- ▶ **Consistência**: Garante que o comportamento dos objetos siga a lógica do domínio do problema.

Para garantir a robustez, devemos verificar se o objeto que está sendo operado é do tipo esperado, evitando erros em tempo de execução.

Tratamento de Tipos Mistos:

Python

```
class Contador:
    def __init__(self, n):
        self.n = n

    def __add__(self, outro):
        if isinstance(outro, Contador):
            return Contador(self.n + outro.n)
        if isinstance(outro, (int, float)):
            return Contador(self.n + outro)
        return NotImplemented

c = Contador(10)
print(c + 5)      # Funciona com inteiro
print(c + "2")    # Erro controlado: TypeError
```

Boas Práticas:

- ▶ **isinstance**: Valida se o operando é compatível antes de realizar a conta.
- ▶ **NotImplemented**: Retorno padrão que informa ao Python que a operação não é suportada para aquele par de tipos.
- ▶ **Flexibilidade**: Permite que a classe interaja com tipos nativos do Python.

Estado e Atributos



O método `__getattr__` é invocado apenas quando o Python não encontra um atributo pelo fluxo normal. Isso permite criar interfaces dinâmicas e flexíveis.

Exemplo: Proxy de Registradores

Python

```
class MapaHardware:
    def __init__(self):
        self.base = 0x4000

    def __getattr__(self, nome):
        # Intercepta qualquer nome de atributo
        print(f"Buscando endereço para {nome}...")
        return f"{self.base}_{nome.upper()}"

hw = MapaHardware()
# 'status' não existe, mas o __getattr__ captura
print(hw.status) # Saída: 0x4000_STATUS
```

Funcionamento:

- ▶ **Sob demanda:** Só é chamado se o atributo não estiver no `__dict__`.
- ▶ **Virtualização:** Permite simular a existência de milhares de atributos sem ocupar memória.
- ▶ **Cuidado:** Não confundir com `__getattribute__`, que intercepta **todos** os acessos.

Para classes que servem primariamente como containers de dados, o decorador `@dataclass` gera automaticamente métodos como `__init__`, `__repr__` e `__eq__`.

Implementação Concisa:

Python

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Telemetria:
    timestamp: float
    sensor_id: str
    leitura: float

dado = Telemetria(171500.0, "DHT11", 25.4)
# Repr amigável gerado automaticamente
print(dado)
```

Vantagens Técnicas:

- ▶ **Redução de Código:** Elimina o "Boilerplate" de inicialização.
- ▶ **frozen=True:** Torna o objeto imutável, garantindo que o estado não mude após criado.
- ▶ **Tipagem:** Incentiva o uso de *Type Hints* para melhor legibilidade.

Ao implementar o método `__getitem__`, sua classe pode ser acessada através de chaves ou índices, comportando-se como uma coleção nativa.

Simulando um Banco de Dados:

Python

```
class HistoricoSensores:
    def __init__(self):
        self._dados = {"temp": 25, "hum": 60}

    def __getitem__(self, chave):
        # Permite acesso via obj[chave]
        return self._dados.get(chave, 0)

log = HistoricoSensores()
print(log["temp"]) # Saída: 25
print(log["lux"])  # Saída: 0
```

Aplicações:

- ▶ **Acesso Intuitivo:** Ideal para buffers de memória ou tabelas de configuração.
- ▶ **Abstração:** Oculta se os dados estão em um dicionário interno, arquivo ou banco de dados.
- ▶ **Extensibilidade:** Pode ser combinado com `__setitem__` para permitir atribuições.

Desvendando Métodos Mágicos no Python: O Guia de Protocolos e Sobrecarga

O conteúdo explora os métodos "mágicos" do Python (identificados pelo prefixo e sufixo ``__``), que definem como os objetos interagem com operações integradas, como cálculos matemáticos, loops de repetição, gerenciamento de contexto (comando ``with``) e representação de dados.

Identidade e Ciclo de Vida



Controle de Instanciação: Diferente do `__init__`, ele controla a criação física do objeto, sendo essencial para implementar o padrão Singleton.

Gerenciamento de Contexto (``with``)



Resumo Rápido		
Método	Propósito	Exemplo de Gatilho
<code>__eq__</code>	Comparação de Igualdade	<code>objeto1 == objeto2</code>
<code>__lt__</code>	Comparação "Menor que"	<code>objeto1 < objeto2</code>
<code>__getattr__</code>	Acesso a atributos	<code>obj.atributo_inexistente</code>

Sobrecarga e Protocolos Customizados



Protocolos de Iteração e Container: Implementar `__iter__` e `__getitem__` permite que sua classe seja percorrida em loops ou acessada via índices.





Prof. Edson José Pacheco

pacheco@ufpr.br

***"Scientists study the world as it is; engineers create the world that has never been.
(Theodore von Kármán)"***